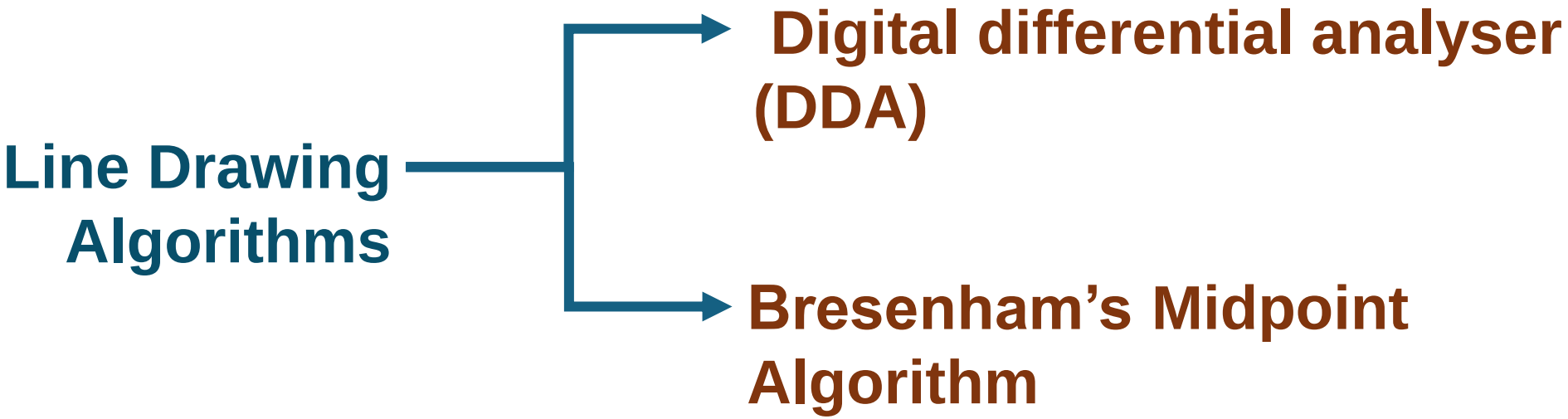
Computer Graphics and Man-Machine Interface

Dr. Manar Donia

CMPn322 - CMPN322 - CMP 422

Line Drawing Algorithms

- **In computer graphics, popular algorithms used to generate lines are-**



Basic Elements

Three basic elements

- Points
- lines
- Vectors

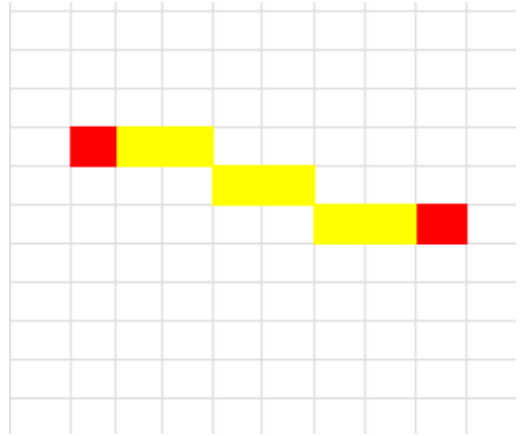
- Points are associated with **locations** in space. The only characteristic that distinguishes one point from another is its position.
- A line is a one-dimensional shape. has a fixed position in 3d space.
- Vectors have magnitude and direction
  - **represent displacements between points, or directions**

Basic Elements

- Points, vectors, and operators that combine them are the common **tools** for solving many geometric problems that arise in
  - Computer Graphics,
  - Animation,
  - Visualization, and Computational Geometry.

COORDINATE REFERENCE FRAMES

- **Line drawing algorithms** for the graphics primitives use the coordinate descriptions to determine the locations of pixels
  - E.g., given the endpoint coordinates for a line segment, it **must calculate the positions for those pixels that lie along the line path between the endpoints.**



- Since a **pixel position** occupies a finite area of the screen
  - the finite size of a pixel must be taken into account by the implementation algorithms.
  - for the present, we assume that each integer screen position references the center of a pixel area.

COORDINATE REFERENCE FRAMES

- **Once pixel positions have been identified** the color values must be stored in the frame buffer
- Assume we have available a low-level procedure of the form

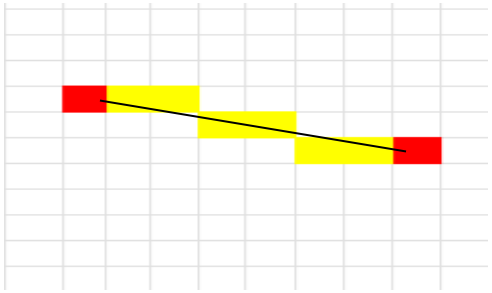
  setPixel

  stores the current color setting into the frame buffer at integer position (x, y), relative to the position of the screen-coordinate origin

  getPixel

  retrieves the current frame-buffer setting for a pixel location.

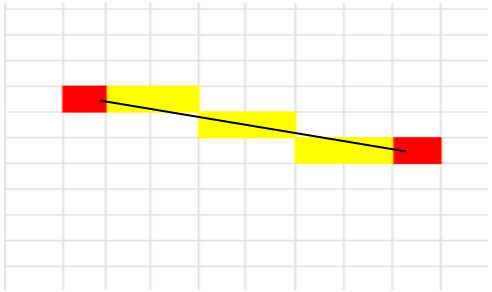
LINE-DRAWING ALGORITHMS



A straight-line segment in a scene is defined by the coordinate positions for the endpoints of the segment.

1. To display the line on a raster monitor, the graphics system must
   - first **project the endpoints** to integer screen coordinates and
   - determine the **nearest pixel positions along the line path** between the two endpoints.
2. Then the line **color is loaded** into the frame buffer at the corresponding pixel coordinates.
3. Reading from the frame buffer, the **video controller plots** the screen pixels.
   - This process digitizes the line into a set of discrete integer positions that, in general, only approximates the actual line path.

LINE-DRAWING ALGORITHMS

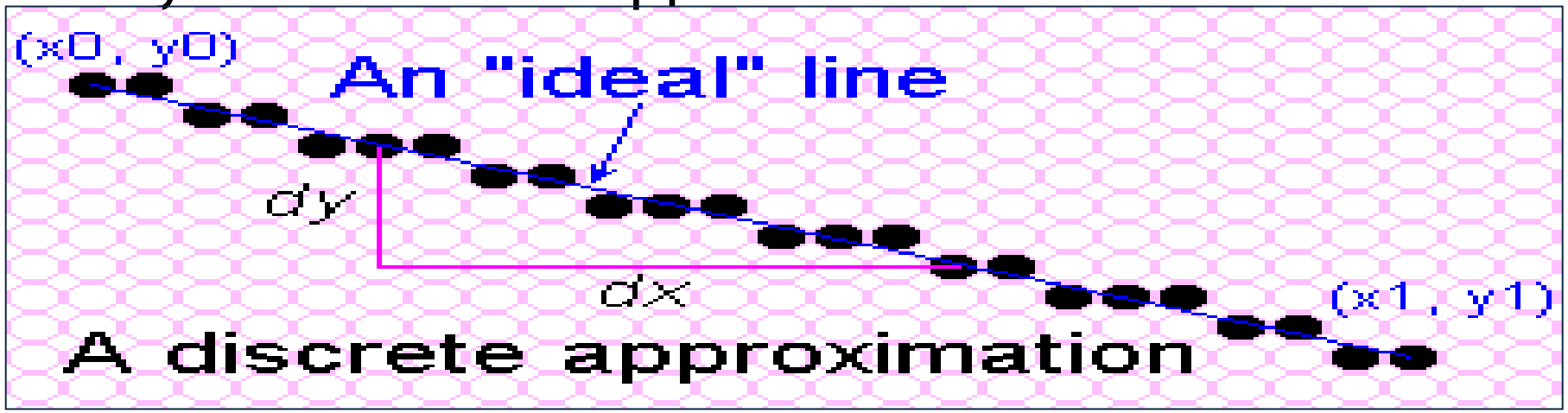


- On raster systems, lines are plotted with pixels, and step sizes in the horizontal and vertical directions are constrained by pixel separations.

- That is, we must "sample" a line at discrete positions and determine the nearest pixel to the line at sampled position.
  - Sampling is measuring the values of the function at equal intervals
- Idea: A line is sampled at unit intervals in one coordinate and the **corresponding integer values** nearest the line path are determined for the **other coordinate**.

Towards the Ideal Line

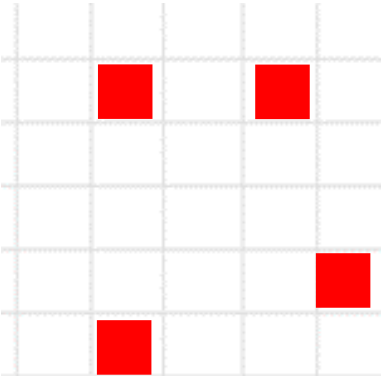
- We can only do a discrete approximation



- Illuminate pixels as close to the true path as possible,
  - consider bi-level display only Pixels are either lit or not lit
- In the raster line alg.,
  - we sample at unit intervals and
  - determine the closest pixel position to the specified line path at each step

Trivial Situations: Do not need LINE-DRAWING ALGORITHMS

- $m = 0 \quad \Rightarrow \quad$ horizontal line
- $m = \infty \quad \Rightarrow \quad$ vertical line
- $m = \pm 1 \quad \Rightarrow \quad$ line $y = \pm x$



Simple Line

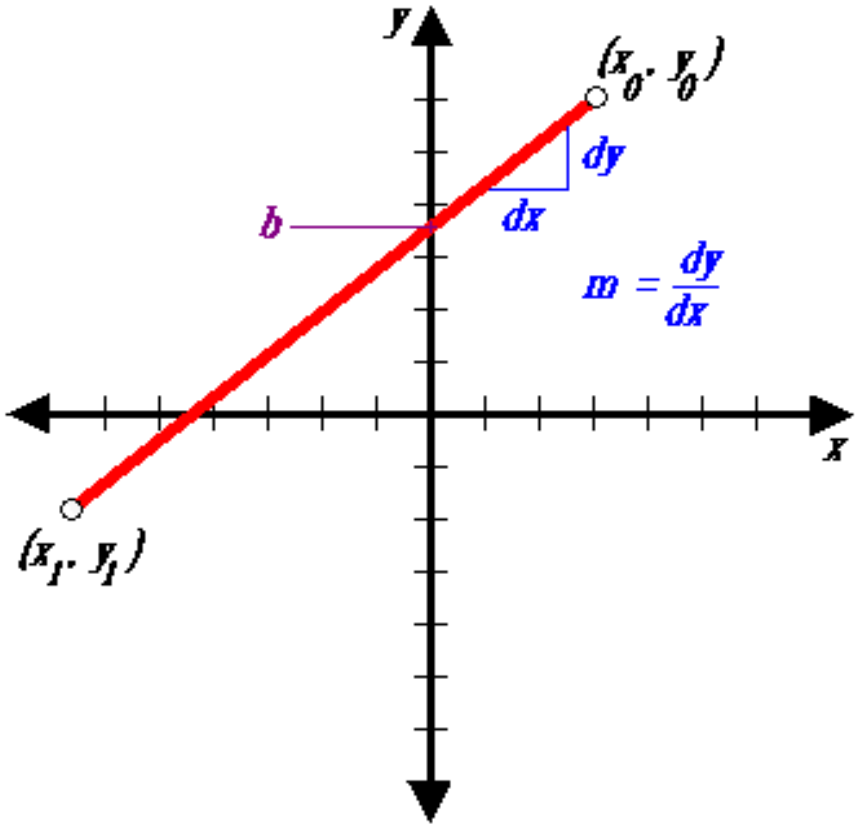
*The **Cartesian slope-intercept equation** for a straight line is :*

$$y = mx + b$$

with *m* as the slope of the line and ***b*** as the ***y*** intercept.

Simple approach:

- increment x, solve for y



Algorithms for displaying lines are based on the Cartesian slope-intercept equation

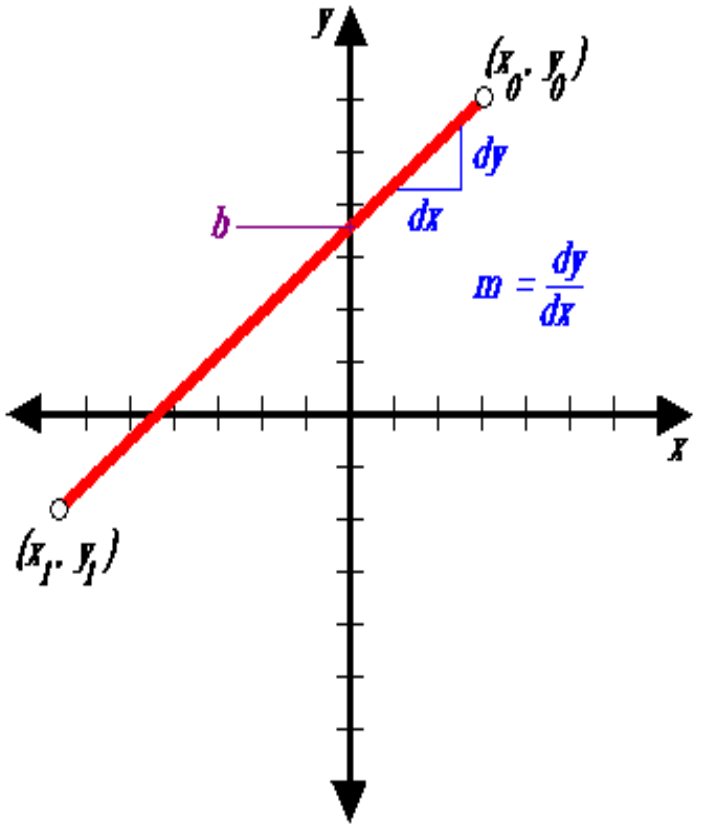
$$y = m.x + b$$

where m and b can be calculated from the line endpoints:

$$m = (y_{end}-y_0) / (x_{end}-x_0)$$

$$b = y_0 - m.\ x_0$$

For any x interval δx along a line the corresponding y interval $\delta y = m.\delta x$

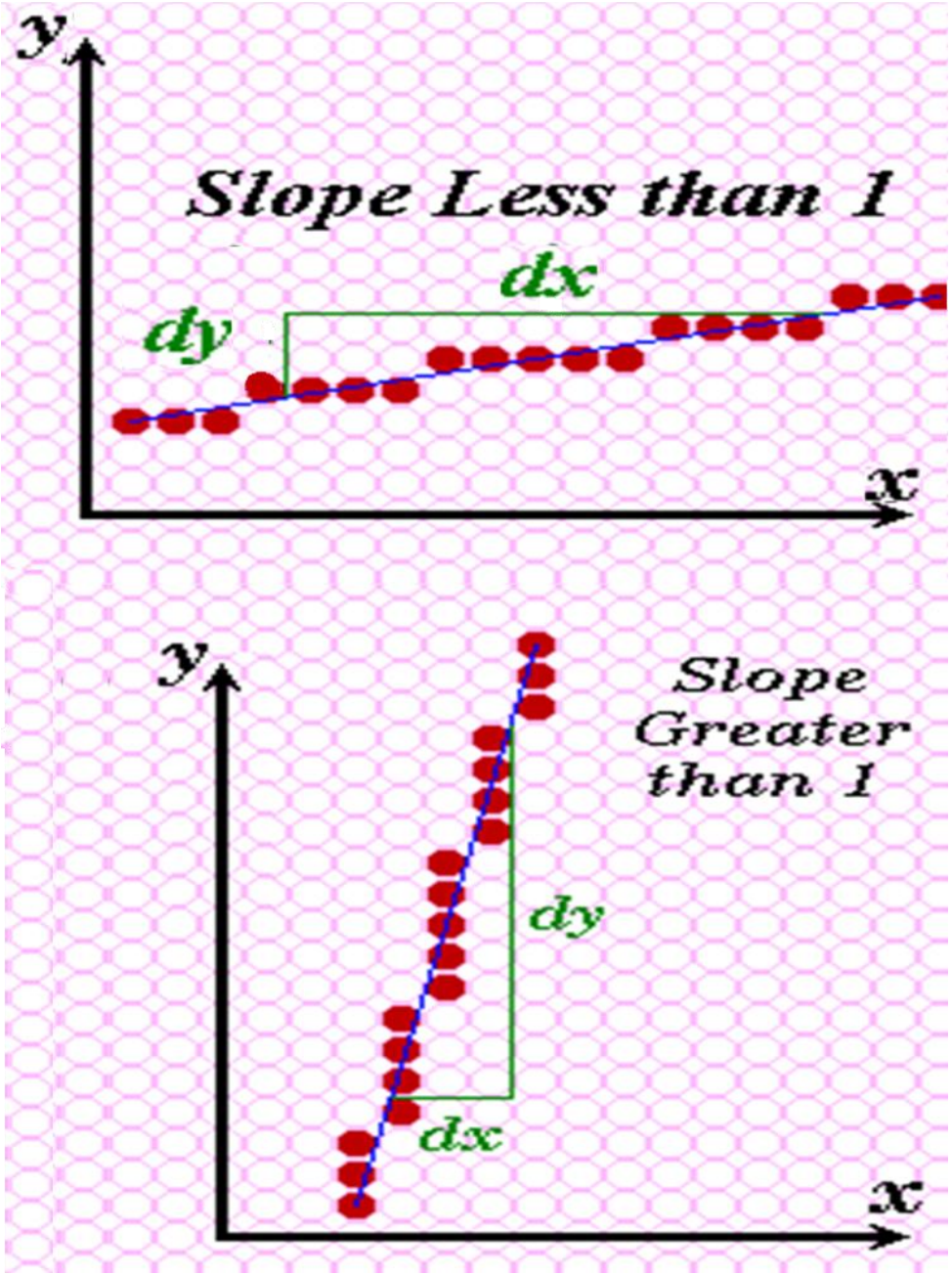


Does it Work?

It works for lines with a slope of 1 or less,

but doesn't work well for lines with slope greater than 1 – lines become more discontinuous in appearance and we must add more than 1 pixel per column to make it work.

Solution? - use *symmetry.*



Line-Drawing Algorithms:

Digital differential analyser (DDA)

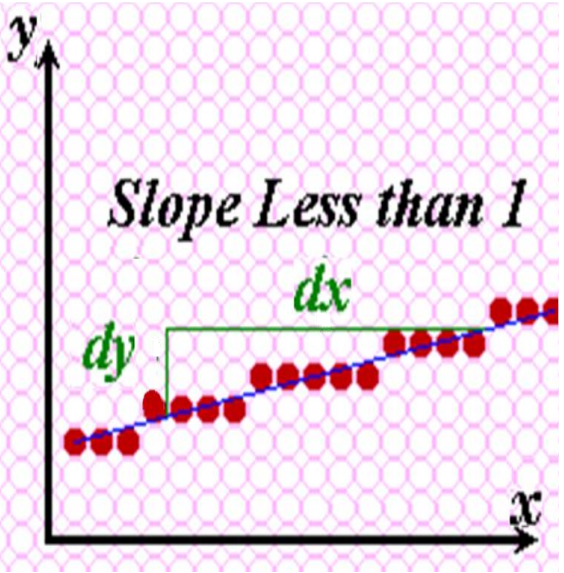
Bresenham’s Midpoint Algorithm

DDA Algorithm

- The digital differential analyser (DDA) is a scan-conversion line algorithm based on using δx or δy.

- **A line is sampled at unit intervals in one coordinate and the corresponding integer values nearest the line path are determined for the other coordinate.**

Line with positive slope



- If m<=1→(dx>dy), Sample at unit x intervals (dx=1)

$$m = \frac{y_{k+1} - y_k}{x_{k+1} - x_k}$$

$$\boldsymbol{x_{k+1} - x_k = 1}$$

$$m = y_{k+1} - y_k$$

$$y_{k+1} = y_k + m$$

Compute successive y values as $y_{k+1}=y_k+m$

$0<=k<=x_{end}-x_0$

- Increment k by 1 for each step
- Round y to nearest integer value.

Line with positive slope

- If m>1→(dy>dx), Sample at unit y intervals (dy=1)

$$m = \frac{y_{k+1} - y_k}{x_{k+1} - x_k}$$

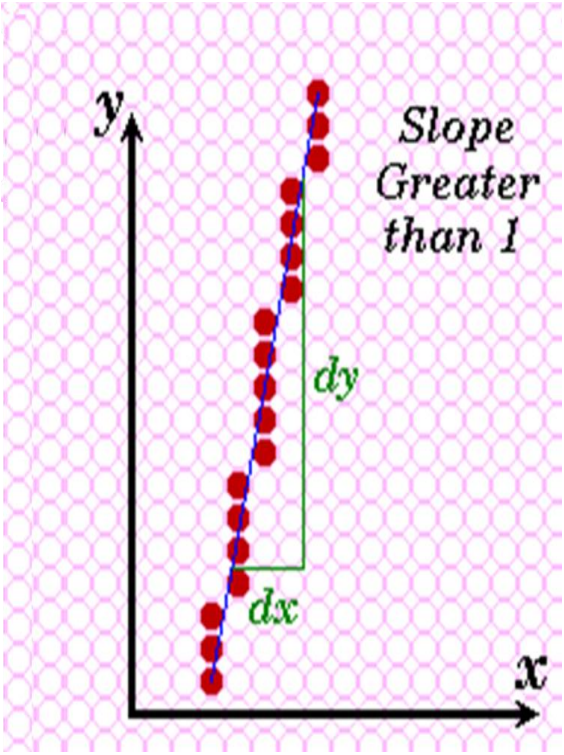
$$\boldsymbol{y_{k+1} - y_k = 1}$$

$$m = \frac{1}{x_{k+1} - x_k}$$

$$x_{k+1} = x_k + \frac{1}{m}$$

Compute successive x values as $x_{k+1}=x_k+1/m$ $0<=k<=y_{end}-y_0$

- Increment k by 1 for each step
- Round x to nearest integer value.



```
inline int round (const float a)  { return int (a + 0.5); }

 void lineDDA (int x0, int y0, int xEnd, int yEnd)
 {
     int dx = xEnd - x0,  dy = yEnd - y0,  steps,  k;
     float xIncrement, yIncrement, x = x0, y = y0;

     if (fabs (dx) > fabs (dy))
        steps = fabs (dx);
     else
        steps = fabs (dy);
     xIncrement = float (dx) / float (steps);
     yIncrement = float (dy) / float (steps);

     setPixel (round (x), round (y));
     for (k = 0; k < steps; k++) {
        x += xIncrement;
        y += yIncrement;
        setPixel (round (x), round (y));
     }
 }
```

// xIncrement =1

//yIncrement =dy/dx=m

//x=x+1
//y=y+m

```
inline int round (const float a)  { return int (a + 0.5); }

 void lineDDA (int x0, int y0, int xEnd, int yEnd)
 {
     int dx = xEnd - x0,  dy = yEnd - y0,  steps,  k;
     float xIncrement, yIncrement, x = x0, y = y0;

     if (fabs (dx) > fabs (dy))
        steps = fabs (dx);
     else
        steps = fabs (dy);
     xIncrement = float (dx) / float (steps);
     yIncrement = float (dy) / float (steps);

     setPixel (round (x), round (y));
     for (k = 0; k < steps; k++) {
        x += xIncrement;
        y += yIncrement;
        setPixel (round (x), round (y));
     }
 }
```

// xIncrement =dx/dy=1/m

//yIncrement =dy/dy=1

//x=x+1/m
//y=y+1

Draw a line from (1,1) to (8,7) using DDA and BLA algorithms.

DDA- Digital Differential Analyser

This case is for slope (m) less than 1. Slope (m) =(7-1)/(8-1) = 6/7 .

S-1: x1=1; y1=1; x2=8; y2=7.

S-2: m=(7-1)/(8-1) = 6/7 which is less than 1.

S-3: As m (6/7) is less than 1 therefore x is increased and y is calculated.

X0=1, xEnd=8, y0=1, yEnd=7
Dx>dy
$x_{k+1}=x_k+1$
$y_{k+1}=y_k+m$

X	Y	(X,Y)
1	1	(1,1)
2	1.86	(2,2)
3	2.7	(3,3)
4	3.57	(4,4)
5	4.43	(5,4)
6	5.29	(6,5)
7	6.14	(7,6)
8	7	(8,7)

Draw a line using DDA Algorithm from (0,0) to (4,6)

This case is for slope (m) greater than 1. Slope (m) =(6-0)/(4-0) = 6/4 .

S-1: x1=0; y1=0; x2=4; y2=6

S-2: m=(6-0)/(4-0) = 6/4 which is more than 1.

S-3: As m (6/4) is greater than 1 therefore y is increased and x is calculated.

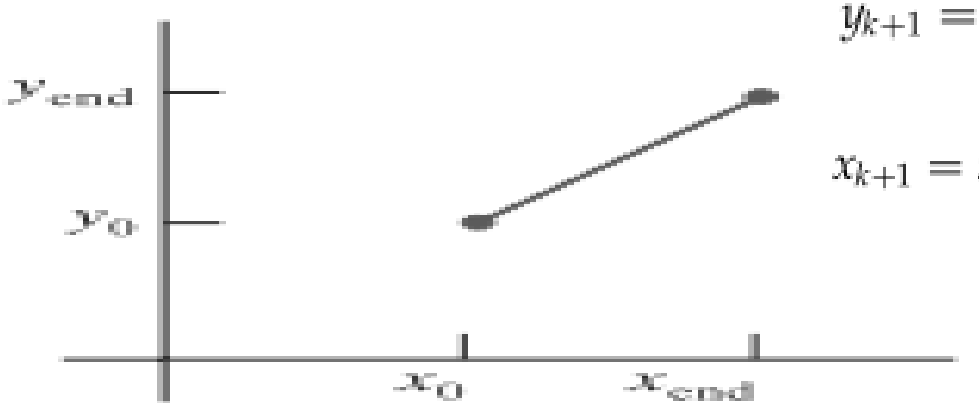
X0=0, xEnd=4, y0=0, yEnd=6

Dy>dx

$y_{k+1}=y_k+1$

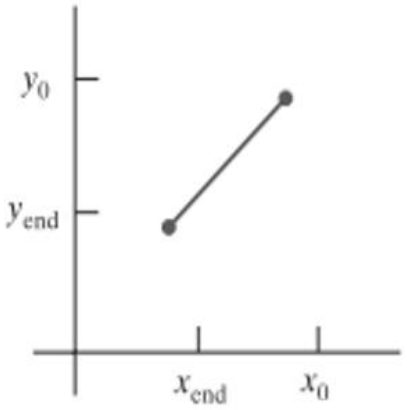
$x_{k+1}=x_k+\frac{1}{m}$

X	Y	(X,Y)
0	0	(0,0)
0.6	1	(1,1)
1.2	2	(1,2)
1.8	3	(2,3)
2.5	4	(3,4)
3.3	5	(3,5)
3.9	6	(4,6)



$$y_{k+1} = y_k + m$$

$$x_{k+1} = x_k + \frac{1}{m}$$



$$\delta x = -1 \text{ or } \delta y = -1$$

$$y_{k+1} = y_k - m$$

$$x_{k+1} = x_k - \frac{1}{m}$$

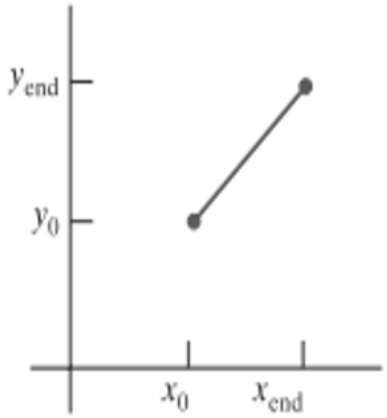
FIGURE 2
Line path between endpoint positions (x_0, y_0) and (x_{end}, y_{end}).

Equations 6 and 7 are based on the assumption that lines are to be processed from the left endpoint to the right endpoint (Figure 2). If this processing is reversed, so that the starting endpoint is at the right, then either we have $\delta x = -1$ and

$$y_{k+1} = y_k - m \tag{8}$$

or (when the slope is greater than 1) we have $\delta y = -1$ with

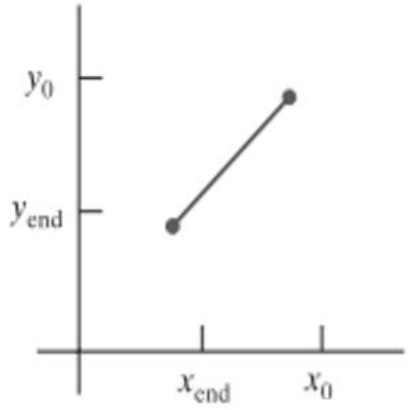
$$x_{k+1} = x_k - \frac{1}{m} \tag{9}$$



$$y_{k+1} = y_k + m$$

$$x_{k+1} = x_k + \frac{1}{m}$$

$$\delta x = -1 \text{ or } \delta y = -1$$



$$y_{k+1} = y_k - m$$

$$x_{k+1} = x_k - \frac{1}{m}$$

Example 3 use DDA algorithm to draw a line from (9,8) to (2,3)

X0=9, xEnd=2, y0=8, yEnd=3
m<=1
Steps =dx=(2-9)= -7
k in range from 0 to 6
$x_{k+1}=x_k-1$
$y_{k+1}=y_k-m$

K	x	y	(x,y)
0	8	7.2	(8,7)
1	7	6.5	(7,7)
2	6	5.8	(6,6)
3	5	5.1	(5,5)
4	4	4.4	(4,4)
5	3	3.7	(3,4)
6	2	3	(2,3)

Digital Differential Analyzer

- **Advantages**

✓It is a simple algorithm to implement.

✓It is a faster algorithm than the direct line equation.

- **Disadvantages:**

✓The floating-point arithmetic implementation of the Digital Differential Analyzer is time-consuming.

✓The method of round-off is also time-consuming.

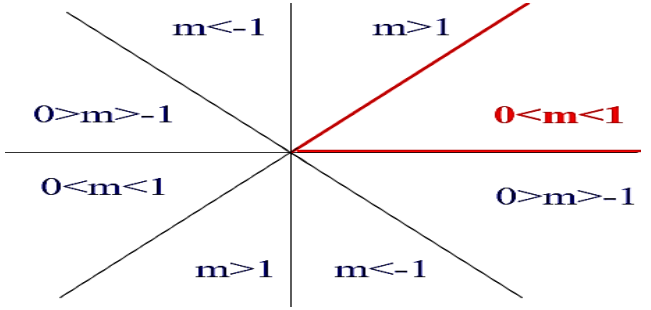
✓The accumulation of round-off error in successive additions of the floating-point increment, however, can cause the calculated pixel positions to drift away from the true line path for long line segments.

Bresenham's line algorithm

- Accurate and efficient
- Uses only incremental integer calculations

The method is described for a line segment with a positive slope less than one

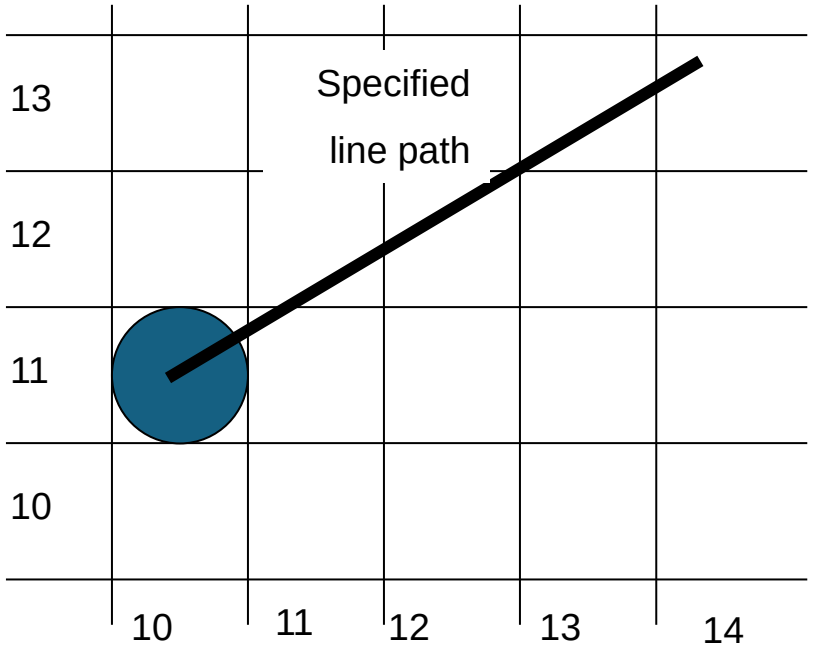
So we work by increment x and calculate y



The method **generalizes** to line segments of other slopes by considering the **symmetry between the various octants** and quadrants of the xy plane

Bresenham's line algorithm

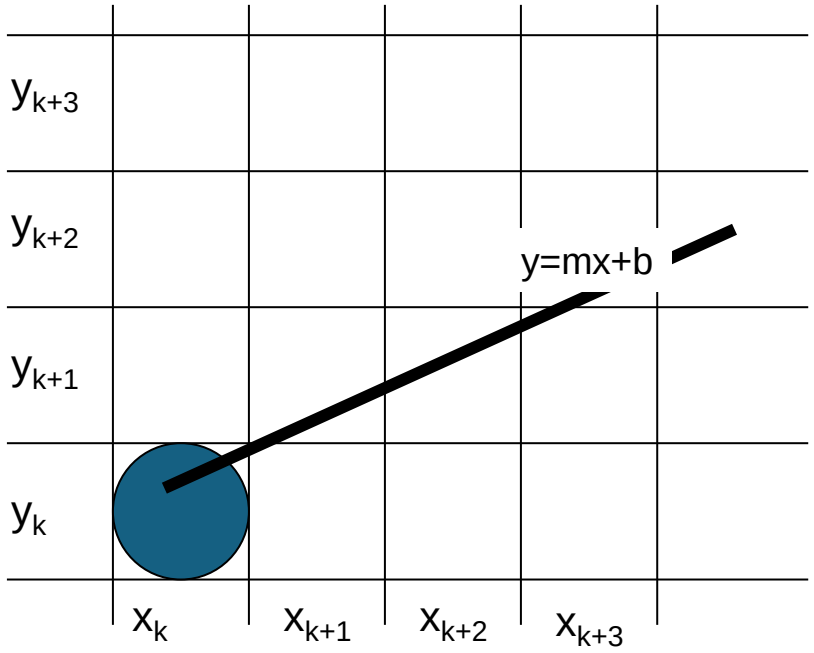
- Draw a line from (10,11) to (14,13)



- Decide what is the next pixel position is (11,11) or (11,12)

Illustrating Bresenham's Approach

- For the pixel position $x_{k+1}=x_k+1$, which one we should **choose: (x_{k+1},y_k) or (x_{k+1}, y_{k+1})**



Bresenham's Line-Drawing Algorithm for I m I < 1

1. Input the two line endpoints and store the left endpoint in (x_0, y_0).

2. Load (x_0, y_0) into the frame buffer; that is, plot the first point.

3. Calculate constants Δx, Δy, $2\Delta y$, and $2\Delta y - 2\Delta x$, and obtain the starting value for the decision parameter ($p_k = \Delta x(d_{lower} - d_{upper})$)

   $p_0 = 2\,\Delta y - \Delta x$

4. At each x_k along the line, starting at k = 0, perform the following test:

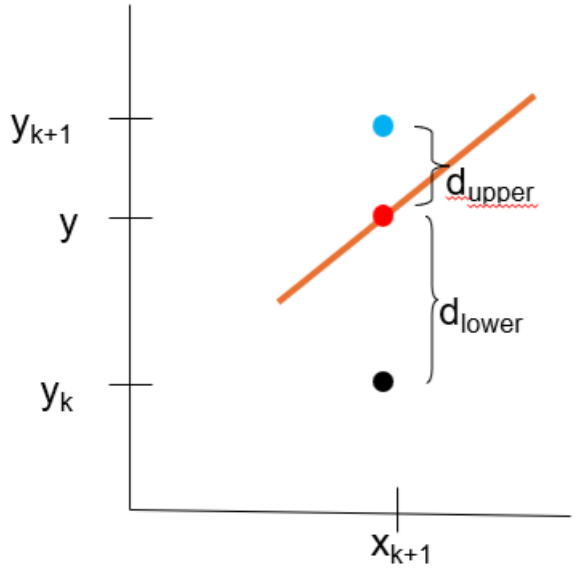
   If $p_k < 0$, the next point to plot is (x_{k+1}, y_k) and

   $$p_{k+1} = p_k + 2\Delta y$$

   Otherwise, the next point to plot is (x_{k+1}, y_{k+1}) and

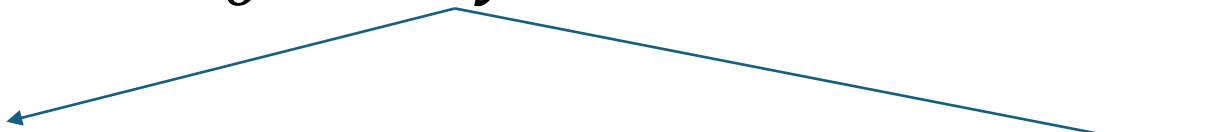
   $$p_{k+1} = p_k + 2\Delta y - 2\Delta x$$

5. Repeat step 4 Δx -1 times.



Successive decision parameter

$$P_o = 2\Delta y - \Delta x$$



P_k>0

Sample x and y

$$P_{k+1} = P_k + 2\Delta y - 2\Delta x$$

P_k<0

Sample x only

$$P_{k+1} = P_k + 2\Delta y$$

Example

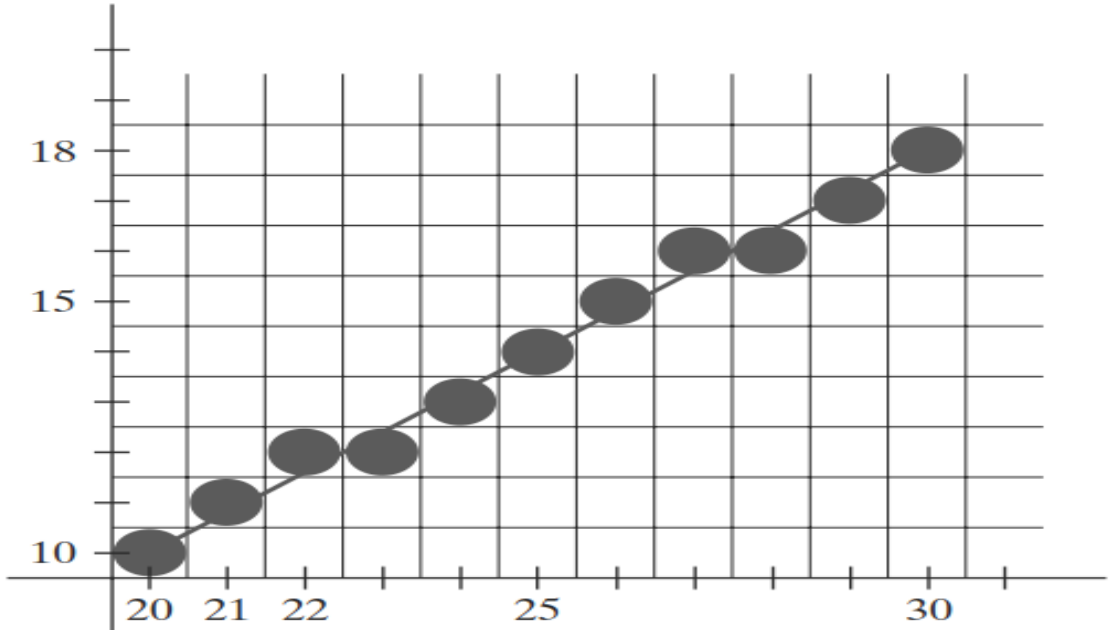
- Draw the line with endpoints (20,10) and (30, 18).
  - **|Δx|=|30-20|=10, |Δ*y*|=|*18-10*|=*8*→ |Δx|> |Δy|**
  - ***p_0* = *2*Δ*y* – Δx=16-10=6**

P_k=>0 **X and Y is incremented**

$$P_{k+1} = P_k + 2\Delta y - 2\Delta x$$
$$P_{k+1} = P_k - 4$$

P_k<0 **X increment and Y is remain**

$$P_{k+1} = P_k + 2\Delta y$$
$$P_{k+1} = P_k + 16$$

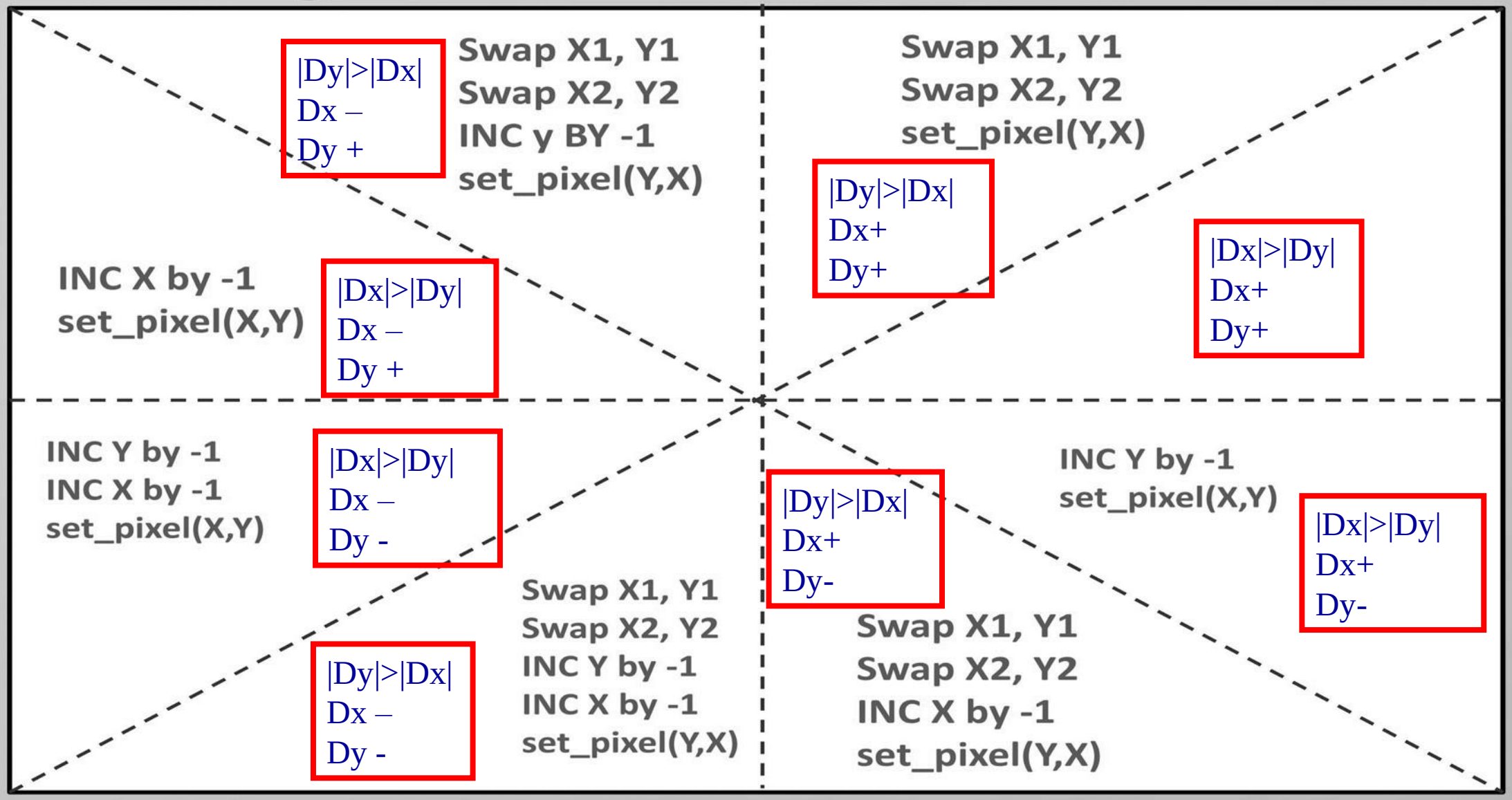


P	X	Y	SetPixel(x,y)
-	20	10	(20,10)
6	**21**	11	(21,11)
2	**22**	12	(22,12)
-2	**23**	12	(23,12)
14	**24**	13	(24,13)
10	**25**	14	(25,14)
6	**26**	15	(26,15)
2	**27**	16	(27,16)
-2	**28**	16	(28,16)
14	**29**	17	(29,17)
10	30	18	(30,18)

Bresenham's algorithm is generalized to lines with arbitrary slope by considering the symmetry between the various octants and quadrants of the xy plane. For a line with positive slope greater than 1.0, we interchange the roles of the x and y directions. That is, we step along the y direction in unit steps and calculate successive x values nearest the line path. Also, we could revise the program to plot pixels starting from either endpoint. If the initial position for a line with positive slope is the right endpoint, both x and y decrease as we step from right to left. To ensure that the same pixels are plotted regardless of the starting endpoint, we always choose the upper (or the lower) of the two candidate pixels whenever the two vertical separations from the line path are equal ($d_{\text{lower}} = d_{\text{upper}}$). For negative slopes, the procedures are similar, except that now one coordinate decreases as the other increases. Finally, special cases can be handled separately: Horizontal lines ($\Delta y = 0$), vertical lines ($\Delta x = 0$), and diagonal lines ($|\Delta x| = |\Delta y|$) can each be loaded directly into the frame buffer without processing them through the line-plotting algorithm.

Dy>dx

Bresenham's Algorithm



First point (3,32) second point(8,38)

$|\Delta x|=|8-3|=5,\ |\Delta y|=|38-32|=6 \rightarrow |\Delta y| > |\Delta x|$

Swap x points with y points

$X_0=32$, $x_{end}=38$ & $y_0=3$, $y_{end}=8$

So

$|\Delta x|=|38-32|=6$, $|\Delta y|=|8-3|=5$

$p_0 = 2\Delta y - \Delta x = 10-6=4$

P_k=>0

X and Y is incremented

$$P_{k+1} = P_k + 2\Delta y - 2\Delta x$$

$$P_{k+1} = P_k - \mathbf{2}$$

P_k<0

X increment and Y is remain

$$P_{k+1} = P_k + 2\Delta y$$

$$P_{k+1} = P_k + \mathbf{10}$$

P	X	Y	SetPixel(y,x)
-	32	3	(3,32)
4	**33**	4	(4,33)
2	**34**	5	(5,34)
0	**35**	6	(6,35)
-2	**36**	6	(6,36)
8	**37**	7	(7,37)
-	38	8	(8,38)

First point (8,32) second point(3,38)

$|\Delta x|=|3-8|=|-5|$, $|\Delta y|=|38-32|=6$ → $|\Delta y|>|\Delta x|$

Swap x points with y points

$X_0=32$, $x_{end}=38$ & $y_0=8$, $y_{end}=3$

So

$|\Delta x|=|38-32|=6$, $|\Delta y|=|3-8|=|-5|$

$p_0 = 2\Delta y - \Delta x = 10-6=4$

P_k=>0

X incremented and Y is decremented

$$P_{k+1} = P_k + 2\Delta y - 2\Delta x$$

$$P_{k+1} = P_k - \mathbf{2}$$

P_k<0

X is incremented and Y is remain

$$P_{k+1} = P_k + 2\Delta y$$

$$P_{k+1} = P_k + \mathbf{10}$$

P	X	Y	SetPixel(y,x)
-	32	8	(8,32)
4	**33**	7	(7,33)
2	**34**	6	(6,34)
0	**35**	5	(5,35)
-2	**36**	5	(5,36)
8	**37**	4	(4,37)
-	38	3	(3,38)

First point (38,3) second point(32, 8)

$|\Delta x|=|32-38|=6,\ |\Delta y|=|8-3|=5 \rightarrow |\Delta x|> |\Delta y|$

$p_0 = 2\Delta y - \Delta x=10-6=4$

P_k =>0

P_k<0

Sample x and y

$$P_{k+1} = P_k + 2\Delta y - 2\Delta x$$
$$P_{k+1} = P_k - \mathbf{2}$$

Sample x

$$P_{k+1} = P_k + 2\Delta y$$
$$P_{k+1} = P_k + \mathbf{10}$$

P	X	Y	SetPixel(x,y)
-	38	3	(38,3)
4	**37**	4	(37,4)
2	**36**	5	(36,5)
0	**35**	6	(35,6)
-2	**34**	6	(34,6)
8	**33**	7	(33,7)
-	32	8	(38,8)

Difference Between DDA Line Drawing Algorithm and Bresenhams Line Drawing Algorithm

	Digital Differential Analyzer Line Drawing Algorithm	Bresenhams Line Drawing Algorithm
Arithmetic	DDA algorithm uses floating points i.e.Real Arithmetic.	Bresenhams algorithm uses fixed points i.e. Integer Arithmetic.
Operations	DDA algorithm uses multiplication anddivision in its operations.	Bresenhams algorithm uses only subtraction and addition in its operations.
Speed	DDA algorithm is rather slowly than Bresenhams algorithm in line drawing because it uses real arithmetic (floating-point operations).	Bresenhams algorithm is faster than DDA algorithm in line drawing because it performs only addition and subtraction in its calculation and uses only integer arithmetic so it runs significantly faster.
Accuracy & Efficiency	DDA algorithm is not as accurate and efficient as Bresenham algorithm.	Bresenhams algorithm is more efficient and much accurate than DDA algorithm.
Drawing	DDA algorithm can draw circles and curves but that are not as accurate as Bresenhams algorithm.	Bresenhams algorithm can draw circles and curves with much more accuracy than DDA algorithm.
Round Off	DDA algorithm round off the coordinates to integer that is nearest to the line.	Bresenhams algorithm does not round off but takes the incremental value in its operation.
Expensive	DDA algorithm uses an enormous number of floating-point multiplications so it is expensive.	Bresenhams algorithm is less expensive than DDA algorithm as it uses only addition and subtraction.


Thank You